PNY Trainings — Cyber Security Department

Web Application Security — Final Project Assignments

Project 1: Map the Target

Reconnaissance & Attack-Surface Mapping
of OWASP Juice Shop

Student: Hamza Iqbal

Environment: Local lab — Docker on Windows 11 (own host)

Table of Contents

1. Objective

2. Scope & Rules of Engagement

3. Methodology

4. Tools Used

5. Target Overview

6. Attack Surface Map

7. Technology Stack Fingerprinting

8. Key Observations

9. Recon Notes

10. Conclusion

1. Objective

The objective of this project was to perform structured reconnaissance against the target web application and produce a complete attack-surface map — cataloguing every page, form, input parameter, and API endpoint — before any vulnerability testing begins. No attack payloads were executed during this phase; the work was strictly limited to mapping, enumeration, and technology fingerprinting.

2. Scope & Rules of Engagement

- **Target —** Target: OWASP Juice Shop v20.3.0, deployed locally via Docker, at http://localhost:3000
- **Environment —** Isolated lab on the tester's own Windows 11 host (Docker Desktop + WSL 2 backend).
- **Boundary —** Every request stayed inside the local lab instance. No external system was touched, consistent with the lab-only scope of the assignment.

3. Methodology

Step 1: Target deployment — pulled the official bkimminich/juice-shop Docker image and ran it with port 3000 mapped; verified the application was live at http://localhost:3000.

Step 2: Manual browsing — walked through every reachable section (home, search, login, register, basket, contact, score-board), noting forms, fields, and URL parameters.

Step 3: Automated spidering — ran OWASP ZAP 2.17.0 (traditional spider + client/AJAX spider, Dev Standard policy) against http://localhost:3000 to enumerate pages and API endpoints.

Step 4: Endpoint cataloguing — expanded the ZAP Sites tree and recorded every page, API route, and input point in a structured table (Section 6).

Step 5: Technology fingerprinting — inspected HTTP response headers, JavaScript bundle names, cookies, and the /rest/admin/application-version endpoint to identify the stack.

4. Tools Used

Tool	Version	Purpose
Docker Desktop	4.95.0	Container runtime for the target application
OWASP Juice Shop	20.3.0 (bkimminich/juice-shop)	Target application under test
OWASP ZAP	2.17.0	Spidering and site-structure

		mapping
Google Chrome + DevTools	155	Manual browsing; header, cookie, and DOM inspection

5. Target Overview

OWASP Juice Shop is a deliberately vulnerable e-commerce web application used worldwide for security training. The deployed instance presents a modern single-page storefront selling juice products, with account management, product search, a shopping basket, product reviews, and a challenge score-board (Figure 1).

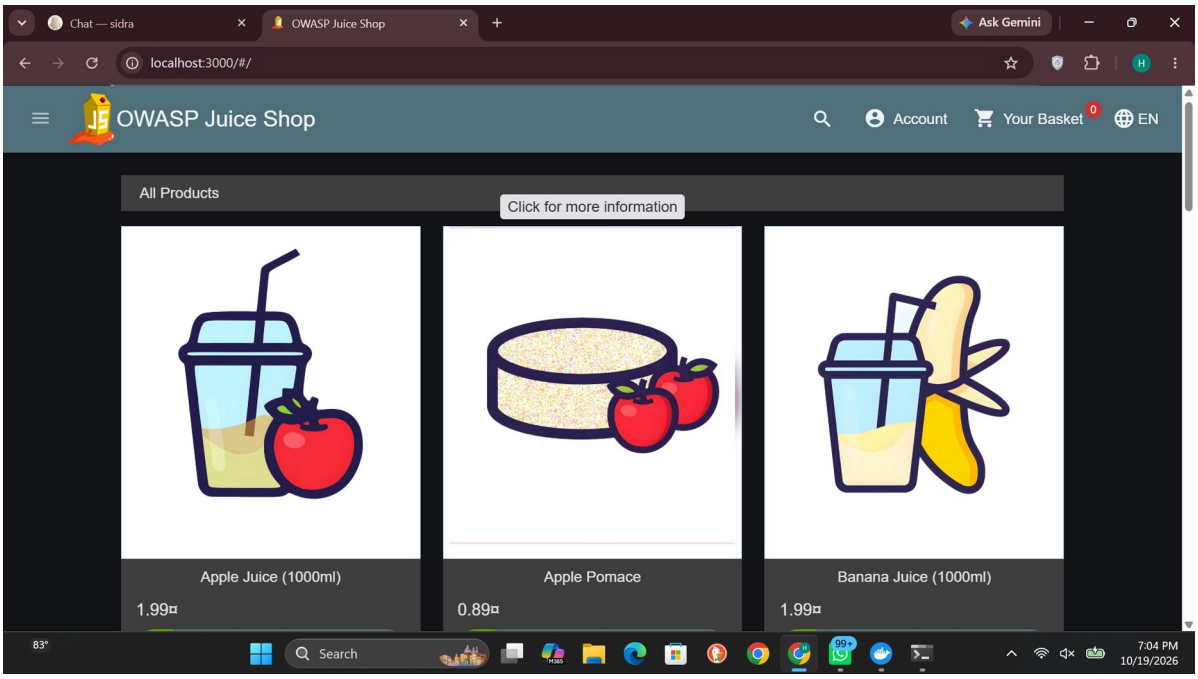


Figure 1 — OWASP Juice Shop storefront running locally at http://localhost:3000

6. Attack Surface Map

6.1 Spider results

ZAP's spider enumerated the full site structure under http://localhost:3000. The Sites tree revealed two API namespaces (/api/* and /rest/*), static asset bundles, and auxiliary files including /robots.txt and the /ftp directory (Figures 2 and 3).

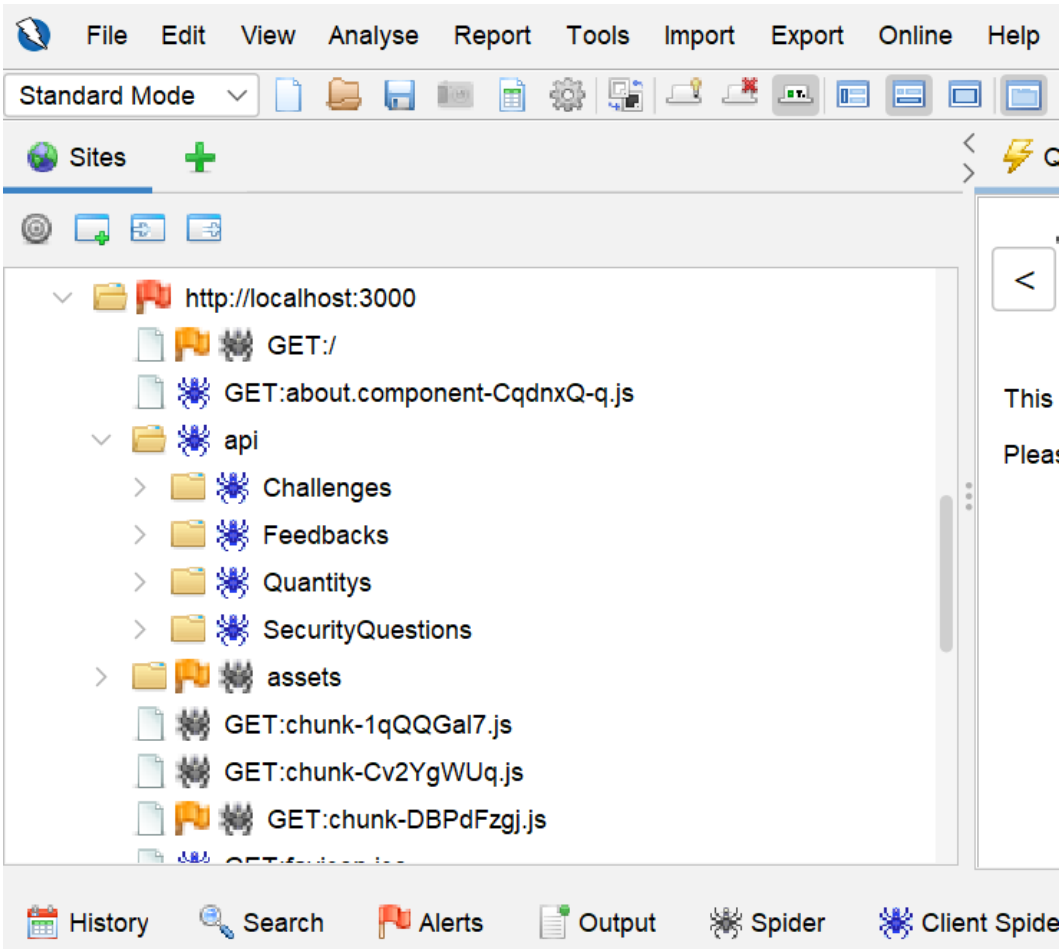


Figure 2 — ZAP Sites tree: /api/ namespace (Challenges, Feedbacks, Quantitys, SecurityQuestions)*

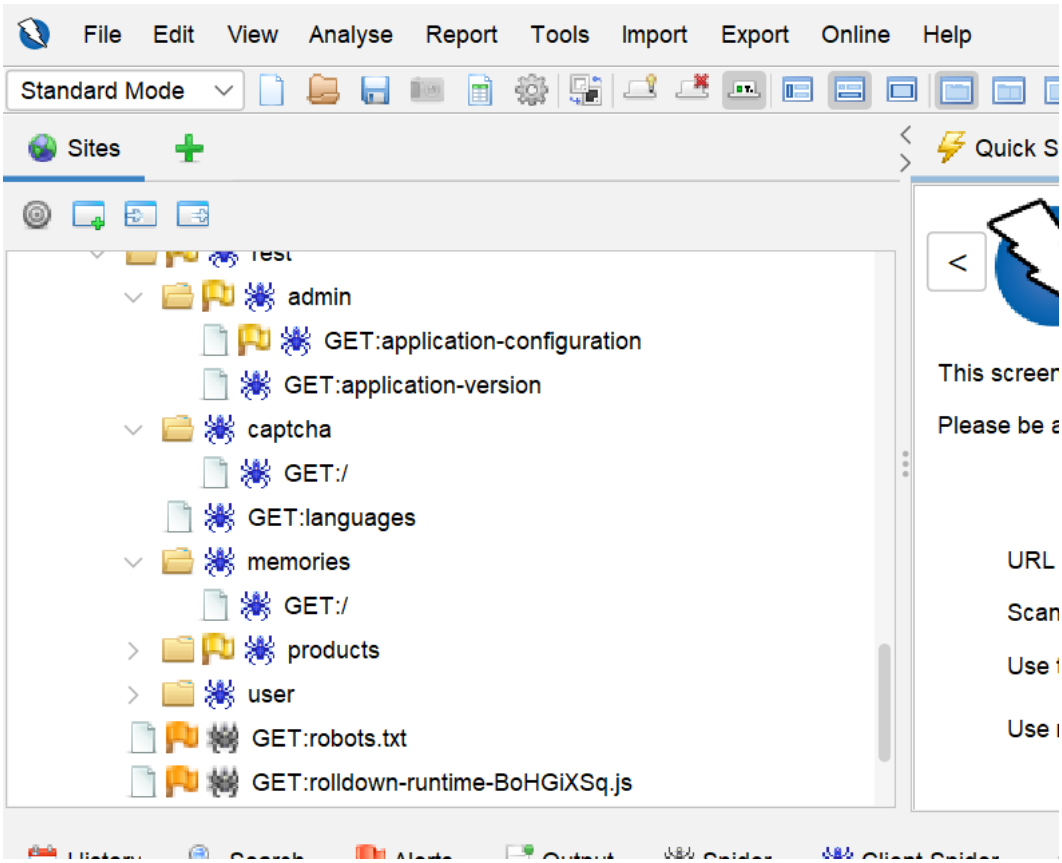


Figure 3 — ZAP Sites tree: /rest/ namespace (admin, captcha, languages, memories, products, user) and /robots.txt*

6.2 Site structure map

The complete attack-surface structure derived from the spider results:

```
http://localhost:3000
├── /                                (Homepage — Angular SPA shell)
├── /#/search?q=...                  (Product search — reflected input)
├── /#/login                         (email, password)
├── /#/register                      (email, password, security Q&A)
├── /#/basket                        (Shopping basket)
├── /#/contact                       (comment, rating, captcha)
├── /#/score-board                   (Challenge progress)
├── /api/
│   ├── Challenges                   (full challenge catalogue)
│   ├── Feedbacks                    (comment, rating)
│   ├── Quantitys                    (stock quantities)
│   └── SecurityQuestions            (registration / recovery)
├── /rest/
│   ├── admin/
│   │   ├── application-version      (version leak — no auth!)
│   │   └── application-configuration (config leak — no auth!)
│   ├── captcha/                     (CAPTCHA service)
│   ├── languages                    (supported languages)
│   ├── memories/                    (user memory wall)
```

```
│   ├── products/                  (search?q= — product backend)
│   └── user/                      (login, register backends)
├── /ftp/                          (file server — confidential docs)
├── /robots.txt                    (crawler directives)
└── /assets/, /media/              (static files)
    ├── main.js, polyfills.js      (Angular bundles)
    └── chunk-*.js                 (Angular lazy chunks)
```

6.3 Endpoint catalogue

Every discovered page, endpoint, and input point is catalogued below. Endpoints marked with * were reachable without authentication.

#	Endpoint / Page	Type	Input / Parameters	Notes
1	/	Page	—	Homepage; Angular SPA shell
2	/#/search	Page	q (query string)	Product search; reflected input — priority for Project 2
3	/#/login	Page	email, password	Authentication form
4	/#/register	Page	email, password, security question + answer	User registration
5	/#/basket	Page	—	Shopping basket
6	/#/contact	Page	comment, rating, captcha	Feedback/contact form — priority for Project 2
7	/#/score-board	Page	—	Challenge progress (JS error loop observed)
8	/api/Challenges *	API (GET)	—	Full challenge catalogue exposed as JSON
9	/api/Feedbacks	API	comment, rating	Feedback backend
10	/api/Quantitys	API	—	Stock quantities

11	/api/SecurityQuestions	API	—	Security questions (registration / recovery)
12	/rest/products/search *	API (GET)	q	Product search backend
13	/rest/user/login	API (POST)	email, password	Login backend
14	/rest/user/register	API (POST)	email, password, security Q&A	Registration backend
15	/rest/admin/application-version *	API (GET)	—	Version disclosure, no authentication
16	/rest/admin/application-configuration *	API (GET)	—	Configuration exposure, no authentication
17	/rest/captcha *	API (GET)	—	CAPTCHA service
18	/rest/languages *	API (GET)	—	Supported languages
19	/rest/memories *	API (GET)	—	User memory wall
20	/robots.txt *	File	—	Crawler directives
21	/ftp *	Directory	—	File server (confidential documents)

6.4 Input-point summary

21 pages, endpoints, and files were mapped. 15+ distinct user-controllable input parameters were identified, concentrated in five areas: product search (q), authentication (email, password), registration (email, password, security question/answer), and the feedback form (comment, rating, captcha). These are the primary candidates for injection testing in Project 2.

7. Technology Stack Fingerprinting

The stack was identified through multiple independent indicators — JavaScript bundle naming, REST URL conventions, and a version-disclosure endpoint — rather than a single header (no X-Powered-By header was returned).

Layer	Technology	Evidence
Frontend	Angular (SPA)	main.js, polyfills.js, chunk-*.js — Angular CLI build output
Backend	Node.js + Express	/rest/* and /api/* REST URL conventions
Database	SQLite	Juice Shop default embedded database
Version	20.3.0	/rest/admin/application-version returned {"version":"20.3.0"} with no authentication (Figure 4)

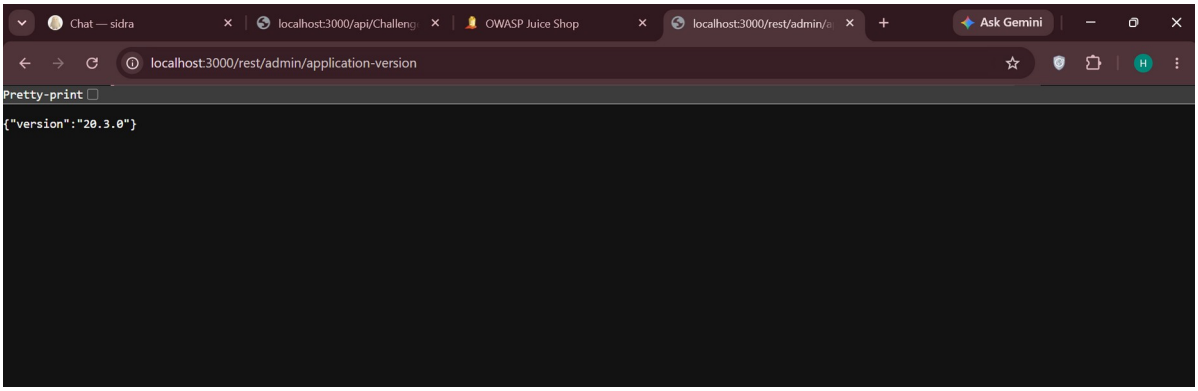


Figure 4 — /rest/admin/application-version discloses the application version without authentication

8. Key Observations

Observation 1 — Unauthenticated admin endpoints. /rest/admin/application-version and /rest/admin/application-configuration respond without any login, disclosing the application version and configuration. Informational at the recon stage; a candidate for Broken Access Control testing in Project 3.

Observation 2 — Full challenge catalogue exposed. /api/Challenges returns every challenge's name, category, difficulty, and description as JSON — useful for planning later test phases.

Observation 3 — Score-board instability. The /#/score-board page enters a JavaScript error loop (39 identical console errors from chunk-Cv2YgWUq.js:4) and never renders — evidence of poor client-side error handling.

Observation 4 — Product API degradation under scan load. After the automated scan, /rest/products/search returned empty results ("0 of 0"); the service recovered after a container restart, indicating fragile backend state handling under load.

Observation 5 — Scanner-triggered challenge solves. ZAP's crawl automatically solved two Juice Shop challenges — "Error Handling" and "Confidential Document" (Figure 5) — confirming the application reacts visibly to malformed input and exposes sensitive files under /ftp.

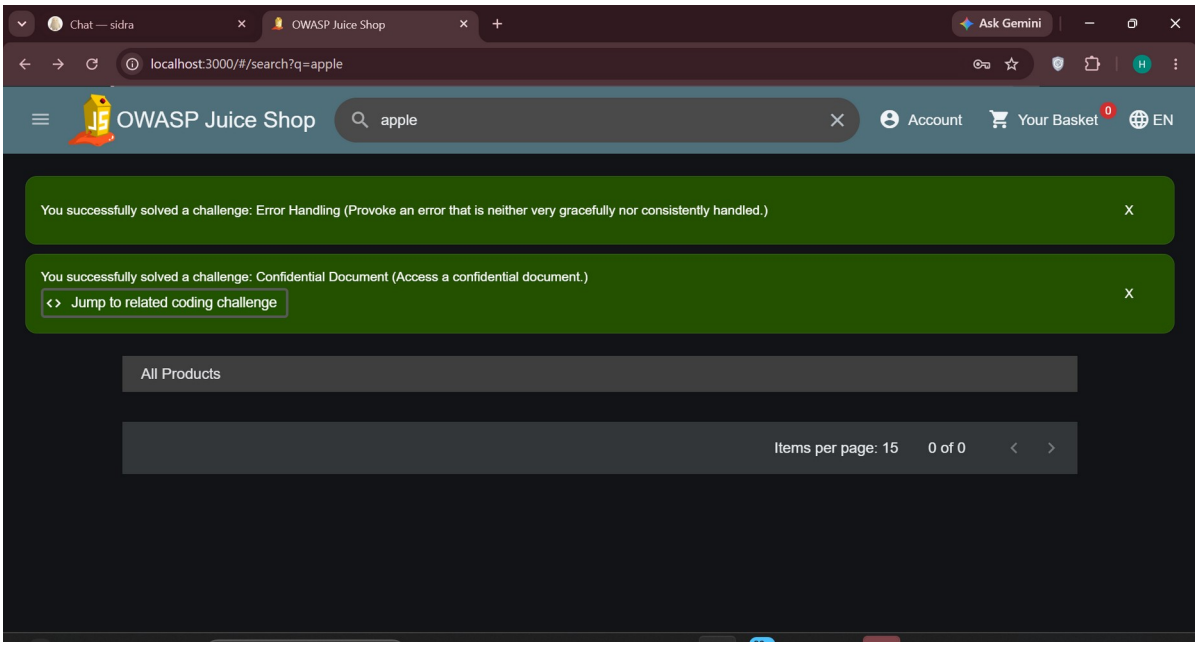


Figure 5 — ZAP's automated crawl triggered two Juice Shop challenges (Error Handling, Confidential Document)

9. Recon Notes

One-page field notes recorded during the exercise.

- Target: OWASP Juice Shop 20.3.0 at http://localhost:3000, running in Docker on my own Windows 11 PC. Lab-only — nothing outside localhost was touched.
- Stack (verified, not guessed): Angular frontend (main.js / polyfills.js / chunk-*.js bundles), Node.js + Express backend (/rest/*, /api/* routes), SQLite database. No X-Powered-By header is sent; the version came from /rest/admin/application-version.
- Mapped 21 pages/endpoints/files and 15+ input parameters. The map lives in the endpoint table (Section 6.2) — the spider found everything the manual walkthrough found, plus the /api/* namespace I had missed.
- Most interesting input points for Project 2: (1) the search box — parameter q is reflected in the URL (/#/search?q=...) and passed to /rest/products/search, classic XSS/SQLi territory; (2) the login form — email and password go to /rest/user/login; (3) the

contact/feedback form — comment and rating are stored and displayed, a stored-XSS candidate.

- Findings worth remembering: /rest/admin/* needs no login (version + config leak); /api/Challenges lists every challenge with descriptions; the score-board page is broken (JS error loop, 39 console errors); the products API went empty right after the ZAP scan and needed a container restart; ZAP's scan alone solved the 'Error Handling' and 'Confidential Document' challenges.
- Lesson learned: map first, attack later. The spider finished in minutes what would have taken an hour by hand, and the unauthenticated admin endpoints would have been easy to miss by browsing alone.

10. Conclusion

Project 1 is complete. The target was fully mapped: 21 pages, endpoints, and files catalogued, 15+ input parameters identified, and the technology stack fingerprinted as Angular + Express + SQLite (v20.3.0). Five notable observations were recorded, including unauthenticated admin endpoints and scan-induced service degradation. This attack-surface map is the foundation for Project 2 (Injection Hunter), with the search parameter (q), the login form, and the feedback endpoints marked as priority injection candidates.